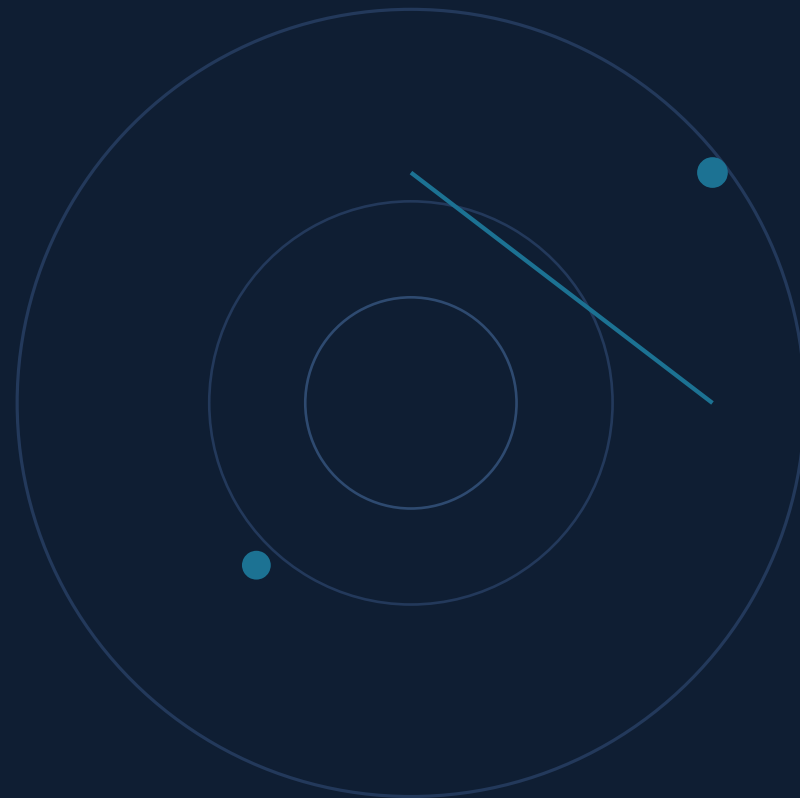


RADAR SW STUDY · PART 3

표적 궤적 모의

구현 방법 및 시뮬레이션 조건

TargetSimCore (C17 / DLL) · TargetSimUI (MFC) · WGS84 좌표변환



목차

과제가 요구한 차례를 그대로 따라간다 .

1 구현 방법

표적을 정의하고 움직인다

- 표적 구조체 정의
- 표적 상태 정보 갱신

2 시뮬레이션 조건

주어진 조건으로 60 초를 돌린다

- 조건값과 실행 순서
- 기동 시나리오
- 결과 확인

3 기타 사항

제공 코드 이용과 결과 설명

- 제공 좌표변환 코드 이용
- 구조체 · 구현 · 결과 설명

과제 1 절 → 2 절 → 3 절 순서 그대로다 . 1 절에서 표적을 정의하고 한 스텝 나아가는 규칙을 만들고 , 2 절에서 주어진 조건으로 60 초를 돌려 LLA 와 그림을 내고 , 3 절에서 제공 코드 이용과 설명 위치를 정리한다 .

요구사항과 진행 순서

1. 구현 방법

- (1) 표적 구조체 정의 — 표적 10 개 · 기동 30 개
- (2) 표적 상태 정보 갱신 — ECEF 좌표계에서
- (+) 동체 속도를 ECEF 성분으로 (추가 고민)

2. 시뮬레이션 조건

- 60 초 · 0.1 초 간격
- 플랫폼 1 + 대함 표적 1 + 대공 표적 1
- 결과를 LLA 로 출력하고 그림으로 표현

3. 기타 사항

- (1) 제공한 좌표변환 코드 이용
- (2) 표적 구조체 정의 설명
- (3) 구현 방법을 코드로 설명
- (4) 시뮬레이션 결과 설명

진행 순서 — 이 자료가 따라가는 차례

1 구조체 정의

1-(1)

2 상태 갱신

1-(2)

3 조건 입력

2 절

4 600 스텝 실행

2 절

5 LLA · 그림

2 절

먼저 표적을 정의하고 (1-(1)), 한 스텝 나아가는 규칙을 세운 뒤 (1-(2)), 주어진 조건을 넣어 600 스텝을 돌리고 결과를 LLA 와 그림으로 낸다 . **제공한 좌표변환 코드는 이 과정 전부에서 그대로 쓴다 .**

전체 구조 — 계산과 화면의 분리

TargetSimCore

C17 / DLL · 화면을 전혀 모른다

Define.h	기본형 · 물리 상수 (참고 소스 + 보완)
matrixCalcLib	행렬 연산 (참고 소스 원본)
CoordinateTransform	WGS84 좌표변환 (참고 소스 원본)
TargetSim	표적 구조체 · 궤적 모의 엔진 (자체 작성)

표적을 한 스텝 나아가게 하는 계산은 전부 여기 있다.
콘솔이든 웹이든 다른 화면을 붙여도 그대로 쓴다.

TargetSimUI

MFC 다이얼로그 · 계산을 하지 않는다

- 입력 칸을 읽어 도 (°) → 라디안 변환
- Core 에 넘기기 전에 값의 형식 · 범위 검사
- 오류는 어느 칸에서 났는지 지목해 반려
- 601 개 결과 표본을 모아서 보관
- LLA 표 · CSV 저장 · 궤적 그림
- 시간 슬라이더로 임의 시점 이동 · 재생

경계가 하나다. UI 는 ST_SimConfig 를 채워 넣고, Core 는 ST_SimSample 을 돌려준다. 그 사이에 오가는 것은 이 두 구조체뿐이다.

설정값과 상태값의 분리

사용자가 넣는 설정값과 매 순간 바뀌는 상태값은 수명이 다르다. 한 덩어리로 두면 어느 값이 계산 중에 바뀌는지 알 수 없다.

입력 — 시작할 때 정해지고 끝까지 안 변한다

```
typedef struct {
    STRUCT_Coord_Lla      st_InitLla;
    STRUCT_Coord_Attitude st_InitAtt;
    FLOAT64               headingSpeed;
    INT32                  nManeuverNum;
    ST_TargetManeuver      st_Maneuver[30];

} ST_TargetInit;
```

개수는 배열이 아니라 숫자로 관리

자리는 최대치 (10 개 / 30 개) 로 미리 잡아두고, 지금 몇 개를 쓰는지만 따로 적는다. 좌석 10 개 버스에 승객 수를 적어두는 것과 같다. 표적을 늘리고 줄여도 메모리 크기가 변하지 않는다.

상태 — 0.1 초마다 바뀐다

```
typedef struct {
    FLOAT64               simTime;
    STRUCT_Coord_Rect     st_PosEcef;    // 계산 기준
    STRUCT_Coord_Rect     st_VelEcef;
    STRUCT_Coord_Lla      st_Lla;       // 출력 + NED 기준
    STRUCT_Coord_Attitude st_Att;

} ST_TargetState;
```

ECEF 가 기준, 위경고도는 파생값

계산은 ECEF 에서 하고, 위도·경도·고도는 매 스텝 ECEF 에서 다시 뽑아 같이 보관한다. 화면 출력과 다음 스텝의 기준 방향 계산에 모두 쓰이기 때문이다.

위치는 LLA, 속도는 V_heading

속도를 **V_heading** 하나, 즉 숫자 하나로 받는다. 그러면 방향은 ? — 자세각 (Yaw·Pitch·Roll) 이 갖고 있다. 기수가 향하는 쪽이 곧 진행 방향이다.

✕ 화살표를 들고 다니며 조금씩 비튼다

속도를 (V_x, V_y, V_z) 세 숫자로 들고 다니면서, 스텝마다 끝을 옆으로 조금씩 밀어 방향을 바꾼다. 민 만큼 빗변이 생겨 화살표가 길어지고, 그게 600 번 쌓인다.

200.00 → 201.45 m/s

1G 로 60 초 선회했을 뿐인데 속력이 0.7 % 늘어난다

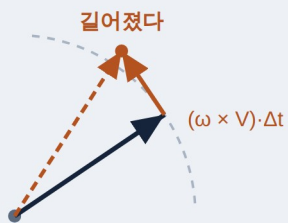
✓ 길이는 숫자로 붙박고 각도만 바꾼다

"이 표적의 속력은 200" 은 숫자 하나로 붙박아 둔다. 스텝마다 자세각에서 기수 방향만 꺼내 거기에 200 을 곱한다.

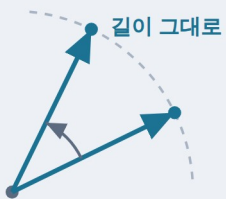
200.000000 유지

몇 번을 돌든 200 은 200 이다

끝을 옆으로 민다



각도만 돌린다



왜 길이가 변할 수 없는가

속도를 만들 때 쓰는 것은 회전 행렬뿐이다. 물건을 돌려도 길이가 변하지 않는 것처럼, 회전 행렬은 벡터의 크기를 바꾸지 못한다.

기동 정보 네 가지

```
typedef struct {
    EN_TurnType  enTurnType;    // 0~3
    FLOAT64      gravityValue;  // [G]
    FLOAT64      startTime;     // [s]
    FLOAT64      endTime;       // [s]
} ST_TargetManeuver;
```

과제가 요구한 네 개를 그대로 답는다.

기동 한 건이 궤적에 걸리는 방식

네 필드가 뜻하는 것

필드	뜻
enTurnType	회전축 — 0 없음 · 1 Roll · 2 Yaw · 3 Pitch
gravityValue	중력 배수 [G], 부호가 회전 방향
startTime	기동이 시작되는 시각 [s]
endTime	기동이 끝나는 시각 [s]

TurnType — 회전축을 그대로 쓴다

- Roll** 옆으로 기울임 · 궤적은 그대로, 자세만 변함
- Yaw** 수평으로 선회 · 위에서 보면 원
- Pitch** 위아래로 선회 · 옆에서 보면 원

구간 안에서만 자세가 돌아간다

startTime 부터 endTime 직전까지 자세각이 일정한 각속도로 돌아가고, 구간 밖에서는 각속도가 0 이다. 기동이 끝나면 그때의 침로 그대로 계속 나아간다.

$$\omega = \text{gravityValue} \times 9.80665 / V_heading$$

예) 2, -1.0, 8.0, 10.4 → 8.0 ~ 10.4 초 동안 1G 좌선회

계산 좌표계 — ECEF

ECEF 는 그냥 더하면 된다

$$\text{새 위치} = \text{이전 위치} + \text{속도} \times 0.1 \text{ 초}$$

지구 중심이 원점인 평범한 x·y·z 직교좌표. 위치 갱신이 벡터 덧셈 한 줄로 끝난다.

위도 · 경도로는 그렇게 안 된다

동쪽으로 1 m 갔을 때 경도가 얼마나 변하는지가 위도마다 다르다. 위로 갈수록 경도선 사이가 좁아지기 때문이다.

동쪽 1 m 이동	경도 변화
적도 (위도 0°)	0.0000090°
위도 60°	0.0000179° (약 두 배)

대신 ECEF → 위경도 변환을 매 스텝 해야 한다. 변환 정확도가 곧 궤적 정확도다

참고 소스의 변환은 같은 계산을 5 번 반복해 위도와 고도를 다듬는다. 반복 횟수가 고정이라 위도가 아주 높거나 고도가 크면 수렴이 모자랄 수 있다. 이 과제 조건 (위도 32 도, 고도 0 ~ 300 m) 에서는 5 번이면 충분하다. 폐형식 (Bowring) 과 맞춰 보니 60 초 내내 수평 차이 0.0003 mm, 고도 유지 오차 0.03 mm 었다.

한 스텝에서 위치가 갱신되는 경로



동체 속도 → ECEF 속도

Yaw 180° 는 " 남쪽을 보고 있다 " 는 뜻이다 . 그런데 그 남쪽이 우주에서 어느 방향인지는 내가 어디에 서 있느냐에 따라 다르다 . 서울에서의 남쪽과 브라질에서의 남쪽은 완전히 다른 방향이다 . 그래서 회전을 두 번 나눠서 한다 .

1 V_body

(200, 0, 0)

기수 방향으로 200 m/s



2 V_ned

자세각으로 회전

내가 선 자리의 북·동·아래 성분



3 V_ecef

위경도로 회전

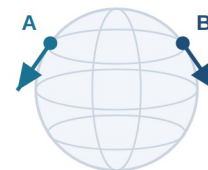
지구 좌표계 성분

```
static STRUCT_Coord_Rect f_Tgt_GetVelEcef(const STRUCT_Coord_Attitude *st_Att,
    const STRUCT_Coord_Lla *st_Lla, FLOAT64 headingSpeed)
{
    STRUCT_Coord_Rect st_VelNed;

    // 참고 소스는 Roll, Yaw, Pitch 순서로 받는다
    st_VelNed = f_Trans_Body_To_Ned(headingSpeed, 0.0, 0.0,
        st_Att->Roll, st_Att->Yaw, st_Att->Pitch);

    // 방향만 옮기므로 기준점은 원점으로 준다
    return f_Trans_Ned_To_Ecef(st_VelNed.x, st_VelNed.y, st_VelNed.z,
        0.0, 0.0, 0.0, st_Lla->Lat, st_Lla->Lon);
}
```

왜 두 번 나눠 도는가



자리마다 '남쪽' 이 다르다

그래서 회전을 두 번 나눠 한다

- ① 자세각으로 V_body → V_ned
- ② 위경도로 V_ned → V_ecef
- ②는 현재 위치마다 다시 만든다

- 두 번째 회전은 표적이 움직일 때마다 다시 만든다

기동과 선회 반경

기동은 위치를 직접 건드리지 않는다. 자세각을 돌리면 다음 스텝의 속도 방향이 바뀌고, 그 결과로 궤적이 휘다.

몇 G 로 돌면 얼마나 빨리 도는가

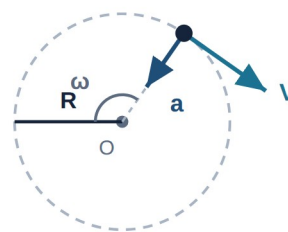
원운동에서 안쪽으로 당기는 가속도는 속도 $\times$ 각속도. Gravity Value 는 이 가속도를 중력의 몇 배로 준 것이다.

$$V \times \omega = n \times g \quad \omega = n \times 9.80665 / V$$

$$\text{선회 반경 } R = V / \omega = V^2 / (n \times g)$$

$$\text{선회 주기 } T = 2\pi V / (n \times g)$$

그림으로 보면



V 속도 (V_{heading})

$a = n \cdot g$ 안쪽으로 당기는 가속도

$\omega = a / V$ 각속도

$R = V / \omega$ 선회 반경

같은 G 라도 V 가 크면 R 이 커진다

속력	G	각속도	선회 반경	한 바퀴	0.1 초에 도는 각도
30 m/s	1 G	18.7 °/s	92 m	19.2 초	1.87°
30 m/s	2 G	37.5 °/s	46 m	9.6 초	3.75°
200 m/s	1 G	2.8 °/s	4,079 m	128.1 초	0.28°
200 m/s	2 G	5.6 °/s	2,039 m	64.1 초	0.56°

같은 G 라도 느린 표적이 훨씬 급하게 돈다.

30 m/s 배는 2G 로 반경 46 m 안에서 돌고,
200 m/s 비행기는 같은 2G 로 2 km 짜리
원을 그린다. 익숙한 감각과 반대다.

적분 방법 — 중점법

같은 0.1 초인데 무엇이 갈리나

0.1 초 동안 속도가 안 변한다고 치고 직선으로 옮긴다. 그 "안 변하는 속도"를 시작(0 초) 시점에서 뽑으면 도는 내내 뒤쳐진 방향으로 간다. 그래서 한가운데(0.05 초)의 자세와 위치로 속도를 만들어 쓴다.

간격을 줄이면 — 200 m/s · 1G 선회 · 60 초 뒤 위치 오차

간격	시작 방향	중점법
0.1 초	19.9 m	0.008 m
0.05 초	9.95 m (1/2)	0.002 m (1/4)

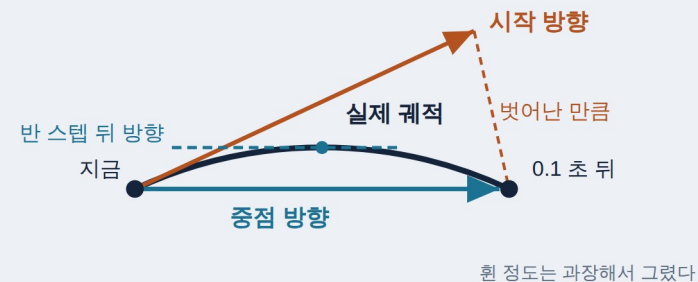
```
halfStep = 0.5 * stepTime;
st_AttMid.Yaw = st_State->st_Att.Yaw + (st_Rate->Yaw * halfStep); // 중앙 자세

st_VelStart = f_Tgt_GetVelEcef(&st_State->st_Att, &st_State->st_Lla, headingSpeed);
st_PosMid.x = st_State->st_PosEcef.x + (st_VelStart.x * halfStep); // 반 스텝만
st_PosMid.y = ... ; st_PosMid.z = ... ;

st_LlaMid = f_Trans_Ecef_To_Lla(st_PosMid.x, st_PosMid.y, st_PosMid.z); // 중앙 위치
st_VelMid = f_Tgt_GetVelEcef(&st_AttMid, &st_LlaMid, headingSpeed); // 중앙 속도

st_Next.st_PosEcef.x = st_State->st_PosEcef.x + (st_VelMid.x * stepTime);
```

한 스텝 안에서 무엇이 갈리나



반경은 두 방법 모두 정확하다. 차이는 궤적이 반 스텝(10 m) 만큼 옆으로 밀리는 것으로 나타난다. 드는 비용은 스텝당 속도 계산 두 번과 위경도 변환 한 번뿐이라, 같은 정확도를 시작 방향으로 내려는 것보다 훨씬 싸다.

한계 하나 — 자세를 roll·pitch·yaw 로 직접 다루므로 pitch 가 $\pm 90^\circ$ 근처면 yaw 축이 진행 방향과 겹쳐(짐벌락) yaw 기동이 궤적을 바꾸지 못한다.

조건값과 실행 순서

주어진 조건	설정값
시뮬레이션 시간 / 갱신 간격	durationTime = 60.0 · stepTime = 0.1 → 600 스텝 , 표본 601 개
플랫폼	32.0°N, 126.0°E, 0 m, 0 m/s (정지)
대함 표적	32.125°N, 126.03°E, 0 m, 30 m/s, Yaw 270° (서쪽)
대공 표적	32.12°N, 126.0°E, 300 m, 200 m/s, Yaw 180° (남쪽)
표적 수	nTargetNum = 2 · 기동 없음 (등속 직진)

실행 순서



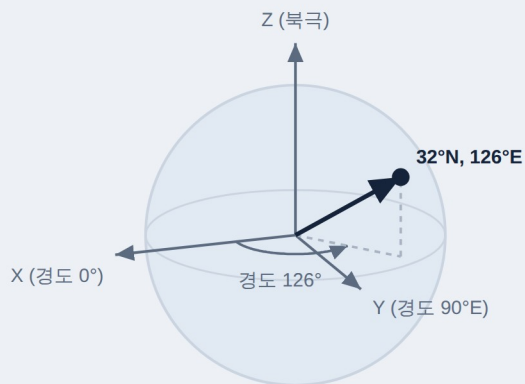
초기값 — 조건이 ECEF 로 바뀐 모습

$t = 0$ 에서 계산된 ECEF 초기값

	ECEF 위치 [m]	ECEF 속도 [m/s]	속력
플랫폼	(-3 182 303.6, 4 380 065.1, 3 360 431.4)	(0, 0, 0)	0
대함	(-3 180 268.5, 4 372 447.8, 3 372 178.2)	(24.2613, 17.6463, 0.0000)	30.000000
대공	(-3 178 301.3, 4 374 556.4, 3 371 868.1)	(-62.5044, 86.0299, -169.3873)	200.000000

이 숫자들을 어떻게 읽나

위치의 부호는 어디서 오나



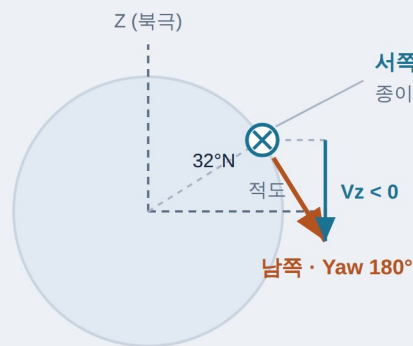
경도 126° 는 90° 를 지났다.

그래서 x 는 음수, y 는 양수.

북반구 32° 라서 z 는 양수.

셋 다 지구 중심에서 잰 거리 [m].

속도의 부호는 어디서 오나



남쪽으로 가면 북극에서 멀어진다.

서쪽 · Yaw 270°
종이면에 수직

그래서 대공 표적은 $V_z = -169.4$.

서쪽은 위도를 바꾸지 않는다.

그래서 대함 표적은 $V_z = 0$.

어느 쪽이든 크기는 200 · 30 그대로.

회전 지점을 기동 시간표로

요구는 위치로 온다

지그재그 기동은 "Lat 32.1 에서 왼쪽으로 90 도 " 처럼 좌표로 주어진다. 그런데 기동 구조체에는 시작 시각과 종료 시각 칸뿐이고 위치를 적을 칸이 없다. 좌표를 시각으로 번역해 줘야 한다.

짚을 것 1 — 도는 동안에도 계속 간다

지정한 위도에서 꺾기 시작하면 90 도를 다 돌았을 때는 이미 선회 반경만큼 더 내려가 있다. 그래서 회전이 끝나는 자리가 지정 좌표가 되도록 반경만큼 미리 꺾는다.

대공 1 기동	위도
꺾기 시작 (트리거)	32.114950°
회전 끝 = 직진 시작	32.110467° ← 여기가 목표

차이 0.004483° = 497 m = 선회 반경 497 m

그래서 한 번 돌려 보고 시각을 읽는다

Core 를 그대로 링크한 도구로 앞에서부터 한 기동씩 본다. 지금까지 정한 기동만 넣고 돌려서, 지정한 좌표를 넘는 순간의 시각을 그 기동의 시작 시각으로 적고 다음 기동으로 넘어간다.

짚을 것 2 — 시작도 끝도 0.1 초 격자 위에만

회전 시간이 0.1 초의 정수배여야 한다. 그래서 45 · 90 · 150 도를 정확히 맞출 수 없다. 부호가 번갈아 상쇄되어 누적 침로 오차는 0.1 도 수준이다.

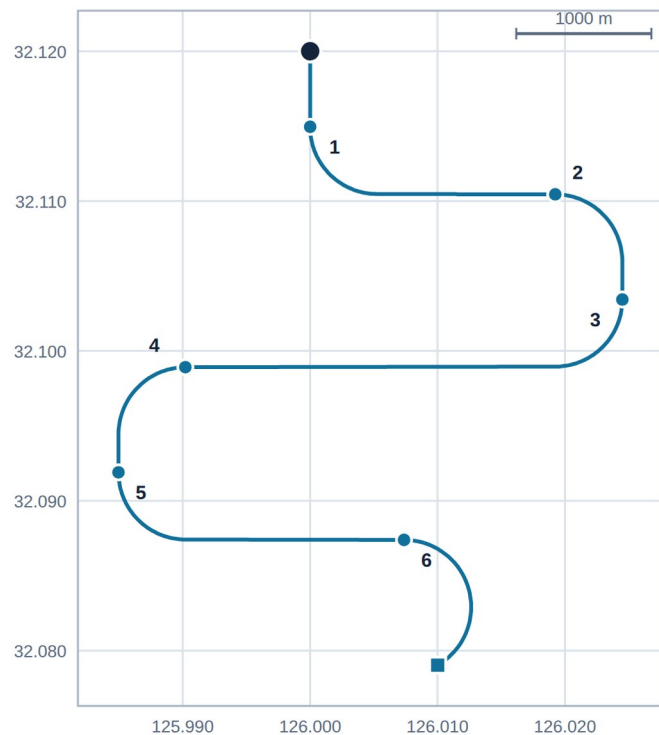
$$G = dYaw[rad] \times V / (g \times T) \quad g = 9.80665$$

표적 · 목표각	실제 회전각	시간	G
대함 · 45°	44.9504°	2.4 s	1.000000
대공 · 90° / 150°	89.90° / 149.83°	3.9 / 6.5 s	8.205060

회전 지점 10 곳이 기동표 10 줄이 된다. 대함 4 줄 · 대공 6 줄. 검출이 0.1 초 격자에 걸려 지정 좌표와 최대 5 m 어긋나는데, 그림으로는 구분되지 않는다.

60 초 시나리오와 검사 결과

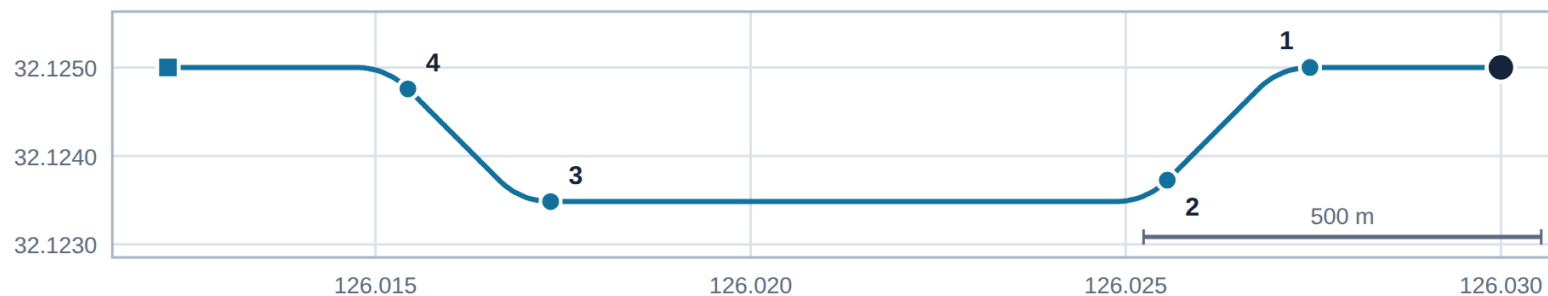
대공 표적 (숫자는 회전이 시작되는 지점)



60 초 시나리오 한눈에

	대함 표적	대공 표적
초기 조건	32.125 / 126.03 / 0 m	32.12 / 126.0 / 300 m
속도 · 침로	30 m/s · Yaw 270°(서)	200 m/s · Yaw 180°(남)
기동	Yaw 선회 4 회 (±44.95°)	Yaw 선회 6 회 (±89.90° 5 + 149.83°)
G · 선회 반경	1.000000 · 92 m	8.205060 · 497 m
스텝당 회전각	1.87°	2.31°
60 초 종점	32.125000 / 126.012232	32.079032 / 126.010006

대함 표적 (좌우 대칭 지그재그)



실제 엔진으로 검사했다. UI 사전검사 (1 ms 격자 · 시간 / 간격 정수배 · 기동 시각 격자 · |G| 한계 · 스텝당 회전각 · 기동 겹침), 설정 검증, 속력 30 · 200 m/s 와 고도 300 m 유지까지 전부 통과. 실패 0 건.

제공 코드 이용과 설명 위치

(1) 제공한 좌표변환 코드 이용

좌표변환과 행렬 코드는 한 글자도 고치지 않고 그대로 쓴다. 프로젝트에 넣은 사본이 원본과 바이트 단위로 같음을 확인했다.

엔진이 부르는 함수	하는 일
f_Trans_Lla_To_Ecef	위경고도 → ECEF
f_Trans_Ecef_To_Lla	ECEF → 위경고도
f_Trans_Body_To_Ned	동체 → NED
f_Trans_Ned_To_Ecef	NED → ECEF (기준점 포함)

CoordinateTransform.c/.h · matrixCalcLib.c/.h

(2) ~ (4) 설명은 어디에 있는가

과제가 요구한 설명	이 자료에서
(2) 표적 구조체 정의	5 ~ 7 장
(3) 구현 방법을 코드로	8 ~ 11 장
(4) 시뮬레이션 결과	12 ~ 15 장

구조체는 설정값·상태값·기동 정보 세 장으로, 구현 방법은 좌표계·속도 변환·선회·적분 네 장으로, 결과는 조건값과 초기값, 기동 시나리오 네 장으로 나눠 설명한다.

원본을 그대로 쓰면서 맞춘 세 가지

인자 순서 — f_Trans_Lla_To_Ecef 는 경도를 먼저, f_Trans_Body_To_Ned 는 Roll · Yaw · Pitch 순서로 받는다. 속도처럼 방향만 옮길 때는 f_Trans_Ned_To_Ecef 에 기준점을 원점으로 준다.

경계 — UI 는 TargetSim.h 한 장만 참조한다. 화면이 쓰는 좌표 변환도 이 헤더로 내보내 원본 함수는 감췄다.

형 이름 — 코딩룰이 요구하는 FLOAT64 · CHAR 를 Define.h 에 더하고, MFC 와 부딪히는 BOOL 매크로는 뺐다.

정리

- 설정과 상태를 나눴다
입력값은 안 변하고 상태값만 변한다 . 개수는 배열이 아니라 숫자로 관리
- 속도를 숫자 하나로 받았다
방향만 자세각에서 매번 새로 만든다 — 회전뿐이라 속력이 변할 데가 없다
- 기동은 네 필드 그대로
구간 [start, end) 동안 각속도가 걸리고 그 밖에서는 직진
- 계산은 **ECEF** 에서
위경도는 그냥 더할 수 없다 . 변환은 참고 소스 코드를 그대로 쓴다
- 동체 속도 → **ECEF** 는 두 번 회전
" 남쪽 " 은 서 있는 자리마다 다른 방향이기 때문
- 한 스텝은 중점법으로
자세와 위치 모두 한가운데 값으로 — 선회할 때 옆으로 밀리는 오차를 상쇄한다
- 회전 지점은 좌표로 , 기동표는 시각으로
엔진을 그대로 돌려 0.1 초 격자 위의 시각을 찾아 옮겼다

다음

결과 출력 및 검증

60 초 시나리오를 UI 에서 돌려 LLA 를 CSV 로 저장하고 , 속력과 고도가 끝까지 유지되는지 확인한다 . 시간 간격을 0.1 → 0.05 로 줄여 오차가 1/4 로 떨어지는지도 함께 본다 .